ACIT 4850 – Enterprise Systems Integration – Assignment 1

Instructor	Mike Mulder (mmulder10@bcit.ca or Discord)
Total Marks	26
Due Date	Wednesday, Feb. 11th at midnight on D2L

This assignment may be completed with your lab partner.

Part 1 – GitLab Upgrade (16 marks)

You should be on version 17.1.0 of GitLab but the latest version is 18.X.X (note: the minor/patch versions may change by the time you do this assignment). You have been tasked with doing the upgrade.

Document your upgrade on a page on your Confluence installation in page called GitLab Upgrade. *If your Confluence license has expired, you may document on the Confluence Cloud, but make sure to include a screenshot showing your expired Confluence install in the dropbox.*

1. Pre Upgrade Planning and Preparation (up to 7 marks)

You will need to review the online GitLab upgrade documents. You can use this site as a starting point: https://docs.gitlab.com/update/

Conduct a pre-upgrade planning activity that documents (at minimum) the following in your report:

- Pre-upgrade assessment, including:
  - Identify your current GitLab version (screenshot) and target version (screenshot)
  - Determining the upgrade path (you can visualize the upgrade path here: https://gitlab-com.gitlab.io/support/toolbox/upgrade-path/)
  - Reviewing the release notes for any key features or fixes
- Backup strategy:
  - What backups do you need to make before the upgrade and where will you store them.
  - Think in terms of the GitLab application and your Cloud provider
- Environment preparation
  - Consider any changes to the operating system or GitLab dependencies (i.e., Postgres, Redis, ElasticSearch, etc).
  - Check the disk space and memory requirements for the target GitLab installation and how much you have available on your VM
- Integration checks

    - Identify any integrations we have made so far with GitLab that need to be maintained after the upgrade (assume we are at least at Lab 5)
- Rollback planning
    - How will you rollback to the previous version if necessary? Consider you backup strategies above.
- Testing Strategy
    - How will you test the upgraded installation. Be specific.
    - For example, how will you check you are on the correct version? What will you check to verify the features/integrations still work?

You will be doing an in-place upgrade during a maintenance window for this assignment.

Q1. Compare this type of upgrade (in-place) to a zero-time upgrade or a migration to a new GitLab installation? What are the pros and cons of each.

Q2. When will you perform the upgrade? Why is it important to consider this for in a production environment?

<u>2. Backup (up to 3 marks)</u>

Based on your plan above, implement your backup strategy.

Take screenshots of all your steps and include it in your report.

<u>3. Upgrade (up to 3 marks)</u>

Perform the upgrade, take screenshots of all the steps in the upgrade.
Make sure to provide you are on your target version of GitLab (include a screenshot showing the version).

<u>4. Testing (up to 3 marks)</u>

Implement your testing strategy from your plan. Take screenshots that verify each of the items in your plan.

<u>Submission</u>

Export your **GitLab Upgrade** page from your Confluence space. Submit it to the Assignment 1 dropbox.

Part 2 – Product Comparison

Document your product comparison on a page on your Confluence installation in page called CI Tool Product Comparison. *If your Confluence license has expired, you may document on the Confluence Cloud, but make sure to include a screenshot showing your expired Confluence install in the dropbox.*

Scenario

If your Lab Group Number is **even** then use Option 1 for this part of the assignment.
If your Lab Group Number is **odd** then use Option 2 for this part of the assignment.

Option 1 - If you Group Number is Even – Continuous Integration in a Growing Startup

You have just joined the IT Team of a small but growing software start-up company. Continuous Integration and Continuous Delivery in the Software Team is currently very ad-hoc and the CTO wants to introduce more formalized tools and processes. The Software Team is using currently using GitLab.com for their private source code repositories (Premium License per developer), JIRA for work management and Confluence as their knowledge base. The production environment for their software is on AWS.

The IT Team is small and stretched thin across setting up equipment for employees, responding to internal help desk requests, managing internal tools and operating the software that the company develops. Since the team is small, they don't want to setup and manage hardware and/or software installations for a CI tool and would therefore need a SaaS option. But to maintain some consistency and good security practices, the tool would need to integrate with their internal LDAP for single sign on and management of users, be easy to configure and manage, provide some level of audit logging (i.e., ability to track usage) and have 2-factor authentication especially for admin user access. It would also be nice if the product had technical support available with a quick turn-around time to questions and address critical issues to ease the burden on your IT Team. Your team has decided that you should assess the following four CI tools:

- GitLab CI (https://about.gitlab.com/)
- Bamboo (https://www.atlassian.com/software/bamboo)
- Travis CI (https://www.travis-ci.com/)
- AWS Code Pipeline (https://aws.amazon.com/codepipeline/)

This start-up is well funded but their CFO is very diligent in managing the company's budget and expenses. The cost of the CI tool must be well defined, both for the one-time costs and training plus ongoing annual costs. So, it must be very clear what these will be to get budgetary approval. It is expected that the CFO would approve up to $300USD per developer per year.

There are two separate Scrum Teams at this start-up company. One is developing a front-end application using React and the another is responsible for two backend applications in Python and Go. The front-end team want to have automated UI tests incorporated into their CI pipelines that exercises all the web pages of their application. The backend team wants strong security analysis for their Python application since it stores sensitive user information and handles credit card transactions. The backend team is new to the Go programming language, so they also want static code analysis as part of the CI pipeline (and ideally this would not require any additional tools to be installed). The test team want to be able to easily deploy all three applications into a test environment. And the CTO wants to move to containerized applications using Docker and Kubernetes so it should provide a Cloud-based deployment platform or support integration with a Cloud-based Kubernetes service (i.e., on AWS). It should also include a Docker container registry (i.e., a place to store the built images) or integration with a 3rd party registry like DockerHub.

Option 2 - If Your Group Number is Odd – Continuous Integration in a Large Enterprise

The Software Development Team at the company you work for are planning to modernize the tools in their development environment. As a member of the Enterprise Development Support Team, your supervisor has tasked you with performing an assessment for one category of development tools and providing a recommendation on which tool to conduct a further "proof of concept" evaluation. For your assigned category, you are provided with 4 products to assess.

The general requirements for any tool in the modernized development environment includes:

- On-premise deployment (i.e., self-hosted) is necessary as the company deals has protected Intellectual Property and works with Government customers who have specific security and data privacy needs
- Cloud SaaS would be nice to have when working with external contractors who don't have access to the internal on-premise environment
- Must support single sign-on integration with Active Directory or LDAP
- Licensing costs must be minimized (and must fit into the manager's budget)
- Has the ability to be deployed in a high-availability manner (i.e., with 99.9% uptime)
- Technical support. Or there must be a significant difference in price that makes the unsupported tool more appealing.

The Software Development Team is significantly modernizing their development processes. They have just migrated from an old source code management system to GitLab and are planning to introduction Continuous Integration to the development process. Your supervisor has tasked you with researching and recommending a CI product from the following tools:

- Jenkins (https://jenkins.io/)
- GitLab CI (https://about.gitlab.com/)
- TeamCity (https://www.jetbrains.com/teamcity). Note: The software teams currently use the JetBrains IDEs which is why this product is on the list.

- Circle CI (https://circleci.com/)

The company is willing to pay a license fee for the product and any option would have to work with the GitLab source code management system and support integrations with GitLab using either plugins or webhooks. However, the Development Manager has an annual budget of $600USD per developer but most of that is being used to pay for their existing GitLab license (which uses ~$350USD of that budget on their current Premium plan)

The Software Development Team has 40 developers and 4 software products in the Ruby, Java, Python and Node/JS programming languages. They would like to leverage feature branching such that each developer's branch would run through the CI pipeline when checked into GitLab. When merged into master, the integrated code would again run through the CI pipeline.

The Test Team already has a large number of automated tests that they would like integrated into the CI pipeline, or even a separate test specific CI pipeline, but they can take several hours to run. They would like the selected tool to have some ability to run their automated tests in parallel to reduce the overall time it takes to run these tests. These tests should be capable of being scheduled to run on specific days and times.

The CI tool would need to be able to integrate with their SonarQube code quality tool and Nexus 3 artifact management tool.

1. Requirements Elicitation (up to 4 marks)

Do this part on your own (with your partner). Do not use AI.

Review the scenario for your Group above. As per Week 3 and 4 materials, identify all the functional and non-functional requirements that are applicable to the scenario. Uniquely number each requirement (same format as in your labs) and categorize them as either mandatory or optional requirements.

Document your requirements in a section called **Requirements Summary** on your Confluence page.

2. Product Comparison (up to 4 marks)

I recommend you use AI for this as it is fairly good at this. But you should check the results for accuracy. Note: You results will be spot checked during the assessment.

Make sure to reference the tool(s) or sites you used for the comparison table.

In a section called **Product Comparison** on your Confluence page, create a detailed comparison table that includes sections for:

- General summary (product name, company name, years available, current version, release frequency, etc.)
- Assessment against the requirements, i.e., does it meet the requirement, not meet the requirement or partially meet the requirement. This must include both mandatory and optional requirements.
- Assessment against at minimum 3 categories of relevant non-functional requirements, such as (but not limited to):
    - Usability
    - Security – make sure to check for outstanding vulnerabilities (i.e., CVE)
    - Availability, i.e., does it provide high availability options
    - Scalability, i.e., can it scale up to more developers and software products
    - Maintainability, i.e., how easy is it to install, manage and upgrade
- Any other criteria that would be relevant for comparison.

The table should look similar in structure to the following:

	Tool 1	**Tool 2**	**Tool 3**	**Tool 4**
General				
Manufacturer	ABC Corp	123 Corp	Big Dev Shop	321 Ltd
Current Release	1.5	4.5.67	3.4A	20.23
…				
Mandatory Requirements				
On-Premise	Yes Only Supports Windows Server	Yes	No Only SaaS Product Available	Yes
…				
Optional Requirements				
…				
Key Non-Functional Areas				
Usability				
…				
And so on				

<u>3. Recommendation</u> (up to 2 marks)

Do this part yourself using the product comparison table above.

In a section called **Recommendations** on your Confluence page, provide a written recommendation on which **two** of the four tools you would choose to conduct a further "proof of concept" evaluation. For the tools not chosen, describe why they were not chosen and whether they would be a viable alternative choice. If there are other better choices available

for this category of tool, identify those and why or why not they would be better. Try to be as objective as possible in the recommendation. **Make sure to refer to your comparison table to justify your recommendations.**

Submission

Export your **CI Tool Product Comparison** page from your Confluence space as a PDF file. Submit it to the Assignment 1 dropbox.

Grading Summary

Part 1 – GitLab Upgrade	16 marks
Part 2 – Product Comparison	10 marks
Make sure your submission is from Confluence and in PDF Format	(-4 marks)
Make sure to include references for your Product Comparison	(-2 marks)
Total	**26 marks**

The Rubric for the above grading breakdown is provided in the Assignment 1 dropbox on the Learning Hub.